CS336 Assignment 2: Systems and Parallelism

Kesavan Ramakrishnan

April 29, 2026

1 Profiling and Benchmarking

1.1 Problem (`benchmarking_script`)

(b) Timings across model sizes

Table 1: Forward, backward, and optimizer-step timings (ms, mean ± std over 10 iters, 5 warmup) on B200 in FP32.

size	forward	backward	optim
small	16.86 ± 0.04	32.12 ± 0.02	7.63 ± 0.02
medium	47.80 ± 0.13	92.78 ± 0.09	17.14 ± 0.12
large	106.85 ± 0.06	208.42 ± 0.18	30.53 ± 0.14
xl	305.85 ± 0.14	571.76 ± 0.39	80.32 ± 0.16

Variability across runs isn't very high, the standard deviation is low and numbers are pretty stable.

(c) Effect of warm-up steps

Table 2: **No Warmup**: Forward, backward, and optimizer-step timings (ms, mean ± std over 10 iters, 5 warmup).

size	forward	backward	optim
small	33.98 ± 51.47	35.51 ± 10.47	7.87 ± 0.13
medium	47.91 ± 0.84	92.57 ± 0.11	16.66 ± 0.11
large	109.19 ± 8.00	208.14 ± 0.39	29.86 ± 0.20
xl	317.26 ± 69.70	570.19 ± 0.55	79.74 ± 0.23

Table 3: **2 iter Warmup**:Forward, backward, and optimizer-step timings (ms, mean ± std over 10 iters, 5 warmup) on B200 in FP32.

size	forward	backward	optim
small	16.74 ± 0.08	32.02 ± 0.03	7.32 ± 0.03
medium	47.50 ± 0.05	92.41 ± 0.04	16.51 ± 0.04
large	106.50 ± 0.06	207.61 ± 0.09	29.87 ± 0.18
xl	295.88 ± 0.23	569.12 ± 0.13	80.85 ± 0.10

With no warmup steps the results are much noisier because the GPU has to warmup and there are latencies that come up from compiling the kernels, kernel setup, etc, making the first runs much longer. As we can see, with no warmup the reported latencies are much higher and have much more variability, but as soon as even 2 warmups are used, the numbers return to normal.

1.2 Problem (`nsys_profile`)

Table 4: Nsight Systems profiling configurations.

model size	context length	batch size	profiled mode
small	256	4	train
small	1024	4	train
small	2048	4	train
large	256	4	train
large	1024	4	train
large	2048	4	train

(a) Total forward pass time vs. Python timing

Table 5: Forward-pass runtime from Nsight Systems compared with Python timing.

profile	nsys fwd (ms)	Python fwd (ms)	difference
small, ctx 256	8.634	10.17	1.536
small, ctx 1024	16.880	37.46	20.58
small, ctx 2048	18.582	90.62	72.038
large, ctx 256	48.389	62.58	14.191
large, ctx 1024	103.621	229.86	126.239
large, ctx 2048	323.612	521.60	197.988

The numbers do not align but this makes sense because the nsys NVTX forward range times the CPU queue window and so it doesn't wait for all the kernels to finish. However, for python side timing, I include a torch.cuda.synchronize() after the forward call and then end timing, which

means the Python side timing measures the total kernel time, which is why the python side timing is higher than the nsys one.

(b) Most expensive CUDA kernel in the forward pass

Table 6: CUDA kernels with the largest cumulative GPU time.

profile	dominant fwd kernel	fwd calls	dominant full-step kernel	full-step calls
small, ctx 256	`cutlass3x_sm100_sgemm_64x64x16_tnn_relu`	24	`cutlass3x_sm100_sgemm_128x64x16_nnn_relu`	72
small, ctx 1024	`cutlass3x_sm100_sgemm_64x64x16_tnn_relu`	28	`cutlass3x_sm100_sgemm_64x64x16_tnn_relu`	28
small, ctx 2048	`cutlass3x_sm100_sgemm_128x128x16_tnn_relu`	7	`cutlass3x_sm100_sgemm_128x128x16_tnn_relu`	39
large, ctx 256	`cutlass3x_sm100_sgemm_64x32x16_tnn_relu`	139	`cutlass3x_sm100_sgemm_64x32x16_nnn_relu`	223
large, ctx 1024	`cutlass3x_sm100_sgemm_128x128x16_tnn_relu`	49	`cutlass3x_sm100_sgemm_64x64x16_ntn_relu`	212
large, ctx 2048	`cutlass3x_sm100_sgemm_128x128x16_tnn_relu`	49	`cutlass3x_sm100_sgemm_128x128x16_tnn_relu`	135

The dominant forward-pass CUDA kernels are all CUTLASS SGEMM kernels. For the shorter-context cases this is a matmul that comes from attention, while the longer-context / larger-model creates wider FFN-shaped GEMMs. The dominant full forward+backward kernel is sometimes the same tile as the forward pass, which indicates that a backward GEMM such as a weight-gradient or activation-gradient matmul is taking the most cumulative GPU time.

(c) Non-matmul kernels with non-trivial runtime

Table 7: Non-matmul CUDA kernels with non-trivial forward-pass runtime.

profile	non-matmul kernels observed	likely component
small, ctx 256	`aten_elementwise_div`	attention + FFN elementwise kernels
small, ctx 1024	`aten_elementwise_div`	attention + FFN elementwise kernels
small, ctx 2048	`aten_elementwise_div`	attention + FFN elementwise kernels
large, ctx 256	`aten_elementwise_div`	attention + FFN elementwise kernels
large, ctx 1024	`aten_elementwise_div`	attention + FFN elementwise kernels
large, ctx 2048	`aten_elementwise_div`	attention + FFN elementwise kernels

For the forward pass there were a lot of elementwise kernels throughout that came from the attn and ffn operations.

(d) Full training step vs. inference fraction breakdown

Table 8: CUDA runtime fraction by kernel category for inference and full training steps.

category	mode	matmul	elementwise / optim	reductions / softmax	movement / indexing
small, ctx 256	forward	78.3%	13.9%	4.2%	3.2%
small, ctx 256	train	64.2%	26.7%	3.6%	5.0%
small, ctx 1024	forward	66.0%	24.6%	7.0%	2.1%
small, ctx 1024	train	64.8%	28.9%	3.8%	2.3%
small, ctx 2048	forward	56.1%	31.8%	8.0%	1.7%
small, ctx 2048	train	56.8%	36.9%	4.4%	1.5%
large, ctx 256	forward	87.0%	8.6%	2.5%	1.9%
large, ctx 256	train	73.5%	20.7%	2.5%	4.1%
large, ctx 1024	forward	76.7%	17.7%	4.5%	1.4%
large, ctx 1024	train	73.0%	21.5%	2.6%	2.5%
large, ctx 2048	forward	66.2%	24.8%	6.6%	1.2%
large, ctx 2048	train	61.0%	33.3%	3.9%	1.8%

Categories: matmul is the CUTLASS SGEMM kernels; elementwise/optim includes add, multiply, divide, sigmoid, pow, sqrt, fill, and AdamW-style vectorized kernels; reductions/softmax comes from reduce max/sum/mean, exp/log, rsqrt, and softmax-related kernels; movement/indexing includes cat/copy, arange, scatter/gather, and indexing kernels.

Generally in comparison to just the forward pass, the full train step is still dominated by matmuls but elementwise and optimizer ops take up a larger percentage in the full train step, making the matmul a smaller percent than in the forward.

(e) Softmax vs. matmul runtime within self-attention

Table 9: Runtime comparison of softmax and matrix multiplication inside self-attention.

profile	softmax (ms)	mask (ms)	scores matmul (ms)	output matmul (ms)	matmul total (ms)	softmax/matmul
small, ctx 256	1.120	0.563	1.011	0.904	1.915	0.58
small, ctx 1024	1.138	0.586	1.029	0.953	1.982	0.57
small, ctx 2048	1.172	0.575	1.010	0.933	1.943	0.60
large, ctx 256	1.858	0.904	1.647	1.555	3.202	0.58
large, ctx 1024	28.851	13.033	1.692	1.629	3.321	8.69
large, ctx 2048	56.977	27.134	1.781	1.681	3.462	16.46

Comparing the softmax to the total time for matmuls, we see that the softmax takes a significant amount of time especially at a bigger model size and longer context length where it dominates. Softmax FLOPs is O(n) while matmul FLOPs is O(n^3) which shows that softmax takes significantly more time than difference in FLOPs.

1.3 Problem (`mixed_precision_accumulation`)

When accumulating in a lower precision (f16), we see that the output of the function is 9.9531 which is inaccurate. However, when doing computation in fp32, the value is precise, and when doing an fp16 operation but accumulating in fp32, so you can operate in a lower precision, accuracy is still recovered (10.0021 vs 10.0001), showing that accumulating in a higher precision preserves accuracy even if the operation is done in a lower precision.s

1.4 Problem (benchmarking_mixed_precision)

(a) Data types under autocast (FP16)

Model parameters dtype: fp32 Output of first feed-forward: fp16 Output of layer norm: fp32 Model's logits: fp16 Loss: fp32 Gradients: fp32

(b) Layer norm sensitivity in mixed precision

With fp16 mixed precision, the layer norm is done in fp32 because the layer norm calculates the mean and variance of a set of values, which can become a big value that can't be stored in fp16, making it sensitive. If we were to do it in bf16 this would be better since we can represent the range of fp32, but we then lose precision because we have less mantissa bits, meaning it will be less accurate.

(c) BF16 mixed precision benchmarking

Table 10: FP32 versus BF16 mixed-precision timing by model size.

size	FP32 fwd (ms)	BF16 fwd (ms)	FP32 bwd (ms)	BF16 bwd (ms)
small	16.63	8.03	31.80	13.50
medium	47.20	16.53	91.86	31.98
large	105.91	29.76	206.26	59.01
xl	291.92	47.16	567.16	104.68

Table 11: Mixed-precision speedup relative to FP32.

size	forward speedup	backward speedup
small	2.07×	2.36×
medium	2.86×	2.87×
large	3.56×	3.50×
xl	6.19×	5.42×

From looking at the table we see that operating in bf16 results in significantly lower runtimes for both the fwd and bwd passes. On top of that, as model size grows we can see that speedups become even more pronounced.

1.5 Problem (memory_profiling)

(a) Memory timeline screenshots

(b) Peak memory per context length

(c) Peak memory under mixed precision

(d) Size of an XL residual-stream activation tensor

(e) Largest allocations in the active memory timeline

(f) Memory per TransformerBlock saved for backward

2 Single-GPU Memory

2.1 Problem (`gradient_checkpointing`)

(a) Memory-optimal checkpointing strategy

(b) Best one-step recomputation strategy for XL

3 GPU Kernels: FlashAttention-2

3.1 Problem (`pytorch_attention`)

3.2 Problem (`torch_compile`)

(a) Compiled vs. uncompiled attention

(b) Compiled vs. uncompiled full Transformer

3.3 Problem (`flash_benchmarking`)

4 Distributed Data Parallel Training

4.1 Problem (distributed_communication_single_node)

4.2 Problem (naive_ddp_benchmarking)

4.3 Problem (minimal_ddp_flat_benchmarking)

4.4 Problem (ddp_overlap_individual_parameters_benchmarking)

(a) Time per training iteration with overlap

(b) Nsight comparison screenshots

5 Optimizer State Sharding

5.1 Problem (`optimizer_state_sharding_accounting`)

(a) Peak memory with vs. without sharding

(b) Effect of sharding on training speed

(c) Comparison to ZeRO stage 1

6 Fully-Sharded Data Parallel

6.1 Problem (`fsdp_accounting`)

(a) Expected memory savings from FSDP

(b) All-gather overlap with the forward pass

7 Analyzing Parallelism Strategies

7.1 Problem (`alternate_ring_all_reduce`)

7.2 Problem (`data_parallel_calcs`)

(a) Backward FLOPs with $N_{\mathbf{DP}}$

(b) Backward communication time with $N_{\mathbf{DP}}$

(c) Maximum $N_{\mathbf{DP}}$ before bottleneck

7.3 Problem (`fsdp_calcs`)

(a) FLOPs for forward and backward

(b) Communication time for forward and backward

(c) Maximum $N_{\mathbf{FSDP}}$ before bottleneck

7.4 Problem (`tp_calcs`)

(a) Backward pass equations for TP

(b) FLOPs for forward and backward with $N_{\mathbf{TP}}$

(c) Communication time for forward and backward

8 Leaderboard

8.1 Problem (`leaderboard`)